Name: Ayan Jafarova

ID: 19824

Microprocessor Systems

Lab Report: Lectures 1-5

GitHub Repository

Task 1: *Via the Arduino IDE use digitalwrite() and blink the onboard LED at 1 Hz. Then use registers to blink the onboard LED at 1 Hz. Estimate and compare the execution time and memory allocation. Using both methods, blink the onboard LED as fast as possible; evidence and explain any differences.*

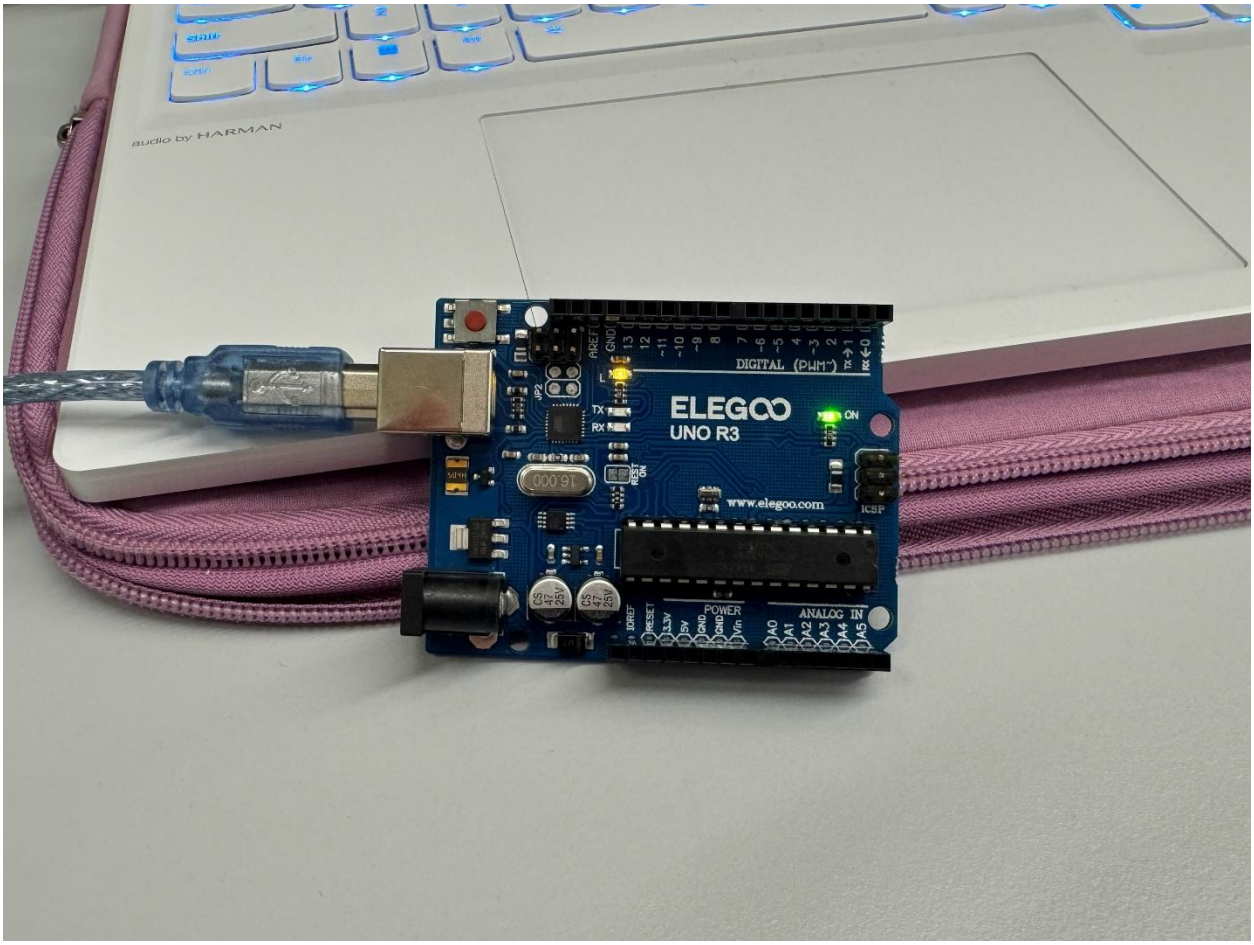


Figure 1 : Arduino UNO R3(ATmega328P) used for the lab tasks loaded with the active code.

Task Overview

The main goal of this lab was to compare the two methods, the digitalWrite() and directly using the registers, for the same action to blink the LED at 1 Hz. The first method is using a high-level language, where the digitalWrite() function is a hardware abstraction layer, which contains the low-level register operations in a more human-readable form. While writing directly to registers leaves no need for the extra functions as it is executing the commands directly.

Method using digitalWrite() at 1 Hz

Using digitalWrite() is convenient way of writing the code for Arduino UNO R3 as it is a human-readable language. When we write "digitalWrite(13, HIGH)" or "digitalWrite(LED_BUILTIN, HIGH)", the first process that happens is that the system looks up the pin number and the port number that are mapped to the digital pin that we use in the code. Then it checks if the pin is configured as OUTPUT with pinMode(), if it is then it sets the pin HIGH, corresponding voltage of 5V, or LOW, equalling the value of 0V(ground). If the pin is not set as OUTPUT with the pinMode() and call the digitalWrite() function, then the LED may appear dim or not ON at all. It is because without setting the pin as OUTPUT we will have an internal pull-up resistor, which will be enabled and act as a current-limiting resistor.

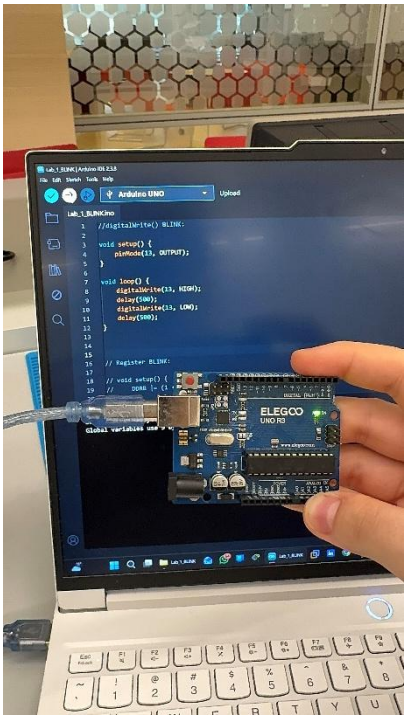


Figure 2 : Arduino UNO R3 microcontroller

Arduino UNO R3 with code written using method with digitalWrite() function and uploaded to the microcontroller.

Method using direct registers access at 1 Hz

The microcontroller ATmega328P which we used for lab tasks controls the I/O through three types of registers per one port. In our case, we are using port B, because the onboard LED is physically wired to PB5.

Register	Name	Purpose	Bit Used
DDR(B0-B7)	Data Direction Register	Configures pin direction as input or output (0 or 1)	DDRB5 (bit 5)
PORT(B0-B7)	Port B Data Register	Sets the output HIGH or LOW (1 or 0)	PORTB5 (bit 5)
PIN(B0-B6)	Port B Input Pins Register	Reads the input state	N/A

To blink the LED directly with the help of the registers, we write DDRB to set the pin 5 as OUTPUT and then toggle PORTB bit 5 directly. Since we are using the registers, we don't need to use extra functions conditions, writing straight to the memory.

Execution time comparison

Method	What happens	Approximate cycle speed
digitalWrite()	Function call triggers the pin lookup to map the port and pins, then checks the configuration of the pin as Output/Input and sets the bit to the correct value.	~50-60 CPU cycles
Direct Register	Writes the value directly to the memory address.	~2-4 CPU cycles

Memory Allocation Comparison

In addition, important inconvenience with digitalWrite() is that it uses more memory compared to direct register manipulation. The significant memory usage happens because when you call digitalWrite(), multiple layers of code run before the actual change, which uses both the flash memory and SRAM. Direct register access compiles 1-2 assembly instructions with no additional memory usage. The difference between them may be small, however on Arduino UNO R3's limited memory of 32 KB (0.5 KB used by the bootloader) for Flash Memory and 2 KB for SRAM, it becomes more convenient and meaningful to use less memory space.

Oscilloscope Evidence

The oscilloscope probe was connected to the Arduino pin 13 and GND pin to measure the period of 1 Hz (1 period per second) showed 1.01 s with the frequency 993 MHz ≈ 1 Hz. The peak voltage was 5.44 V, and the HIGH level was 5.12 V. And the waveforms on this timescale show delay width was ≈ 500 ms proving the execution time of 1 Hz.

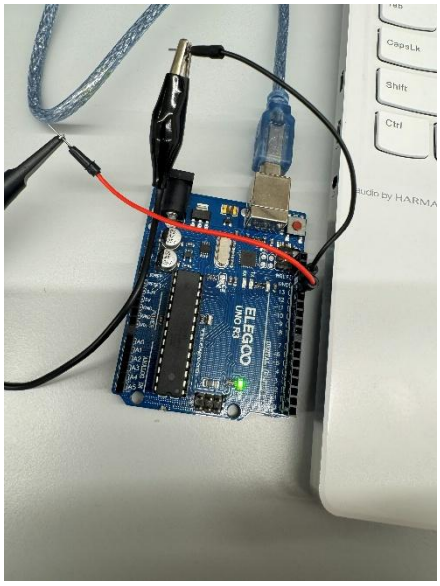


Figure 3 : Oscilloscope probe connected to pin 13 (PB5) and GND

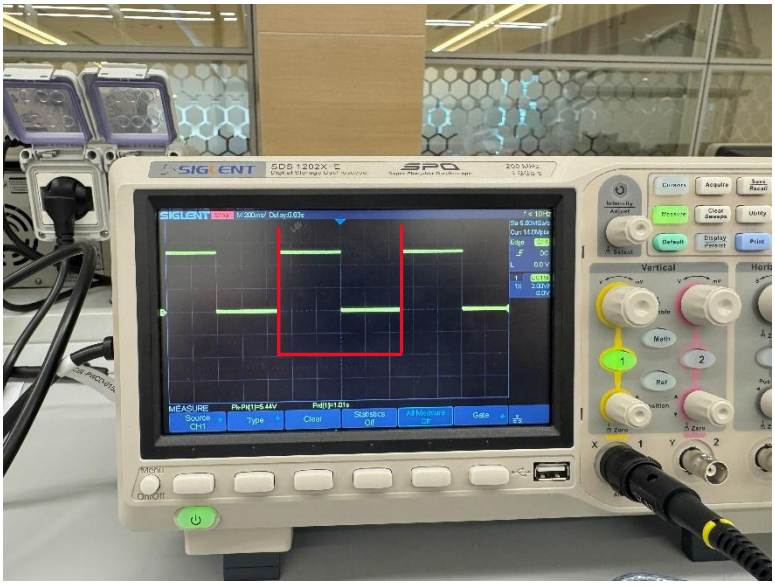


Figure 4: Oscilloscope value for 1 cycle per second = 1 Hz square wave

Maximum Speed Blink

If we remove all delay functions and have the program write straight HIGH and LOW in a loop, then the frequency is hard to detect with human eye, as now the speed of the operations depends only on the number of CPU cycles needed for each level-toggle. In register-level programming, the XOR (^=) toggle flips the current level of the bit 5, and it is used to combine the separate HIGH and LOW states. By using this method, since 1 full BLINK takes 2 instruction loops, we get 1 instruction per loop, which alternates the LED outcome and each toggle takes a few (~2-4) CPU cycles.

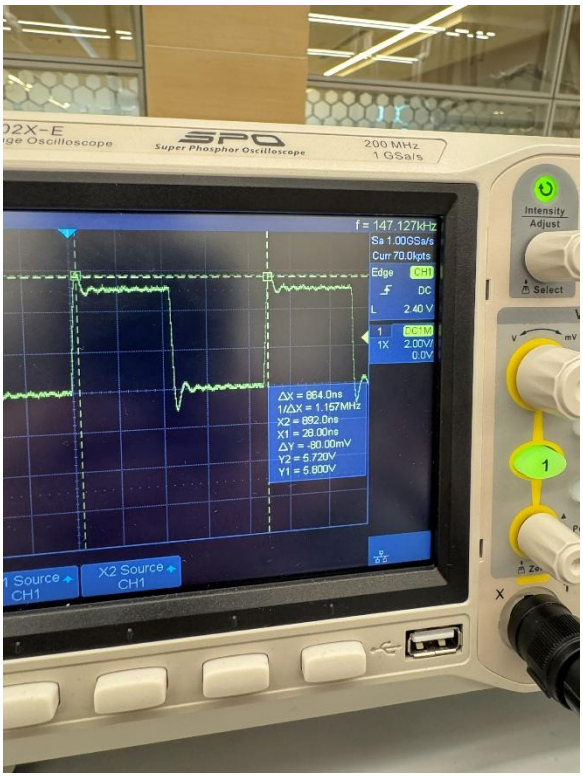


Figure 5: The Fast BLINK using register level language; 1 period (ΔX) = 864.0 ns (nano)

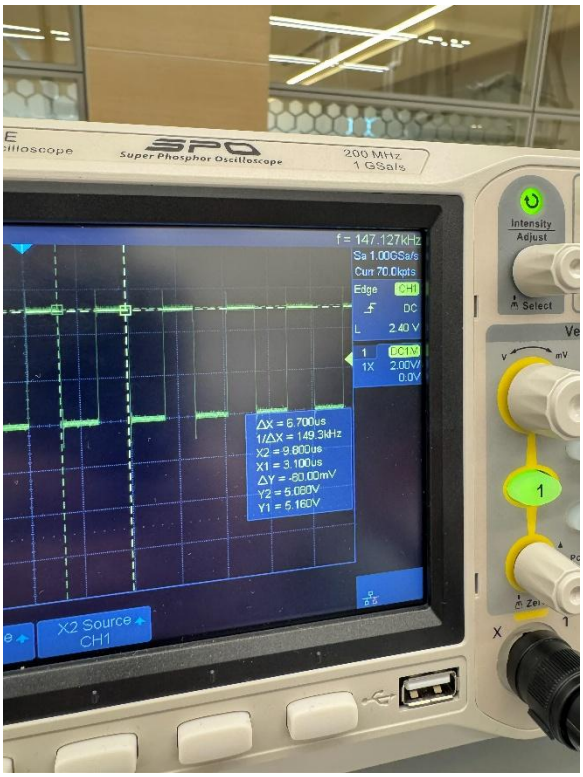


Figure 6: The Fast BLINK using digitalWrite() without delay function; 1 period (ΔX) = 6.700 µs (micro)

Task 2: *Create a serial interface for subtracting two numbers using CPU registers only, store the result in a register, and output all register values. You must then examine the SREG status register and clearly identify the state of the Zero (Z) and Carry (C) flags after the subtraction. Evidence the SREG behaviour and explain why these flags are set or cleared for this operation, with particular emphasis on how the AVR represents underflow in unsigned subtraction and how the Carry flag behaves during subtraction.*

Task Overview

This task asks for application of a serial interface on the Arduino UNO R3, which will accept two unsigned integers as an input and subtracts them using the CPU registers, operating directly on them, then stores the result and Status Register value. Afterwards, it prints all register values, the Zero (Z) and Carry (C) flags to the serial monitor.

The AVR Status Register (SREG)

SREG is an 8-bit register at I/O address 0x3F. There are three different outcomes possible: if A=B => Z=1, C=0; if A>B => Z=0, C=0 and if A<B => Z=0, C=1. The Z flag occupies bit 1 of SREG and the C flag occupies bit 0, to get the result in C++, we need to move to the bit value to the right. Z = ((SREG >> 1) & 1) and C = (SREG & 1).

Bit	Flag	Set to 0	Set to 1
1	Z	The result is any number other than zero	If the result of the arithmetic operation is exactly zero
0	C	The result fits in 8 bits and no borrow is needed (A>=B)	If (A<B) then a borrow is needed as this operation is causing unsigned underflow

SREG Bit Layout

Bit	7	6	5	4	3	2	1	0
Flag	I	T	H	S	V	N	Z	C

How the Code Works

The code reads two integers A and B from the input line using sscanf(), then assigns them to unit8_t for the 8-bit unsigned arithmetic and enters an inline assembly block using the asm volatile directive. The volatile keyword is crucial, as it prevents the compiler from optimizing the assembly block or reorder it in any way.

Value of int A copies into r20, value of int B into r21, then the subtraction process happens, it subtracts int B value from int A value and writes back to r20, which stores the result of the subtraction (r20 = r20 – r21), this immediately updates the SREG flag bits (Z and C). The SREG is captured on the next instruction before any other instruction modifies the flags. And at the end the clobber list informs the compiler that these registers are modified inside the assembly block, so it does not rely on their values anywhere else.

The Program Flow

Serial input	Two unsigned integers are entered with a space between them
⇓	
Parse and assign	sscanf() extracts A, B and assigns them to unit8_t
⇓	
mov r20, a	Loads value of A into register r20
⇓	
mov r21, b	Loads value of B into register r21
⇓	
sub r20, r21	r20 = A – B and the SREG values are updated
⇓	
IN r22, SREG	Capture all needed SREG flags before any other instruction modification
⇓	
Result	Result and SREG bytes sent to the output on the Serial Monitor

Serial Monitor

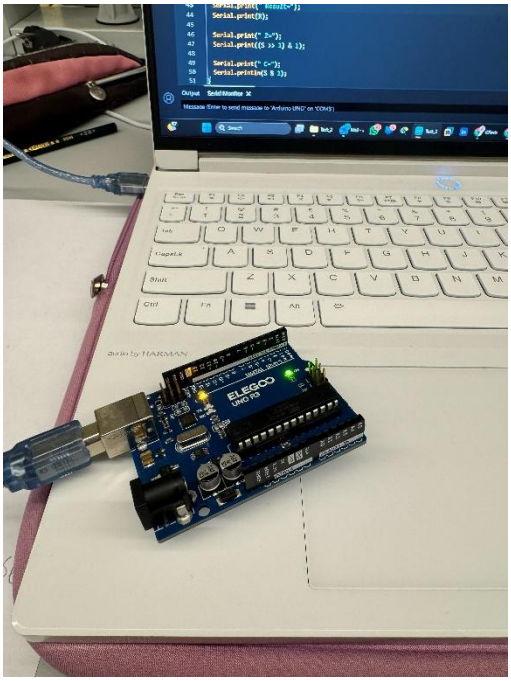


Figure 7 : Arduino UNO R3 running the code for Task 2 uploaded on it and with Serial Monitor active

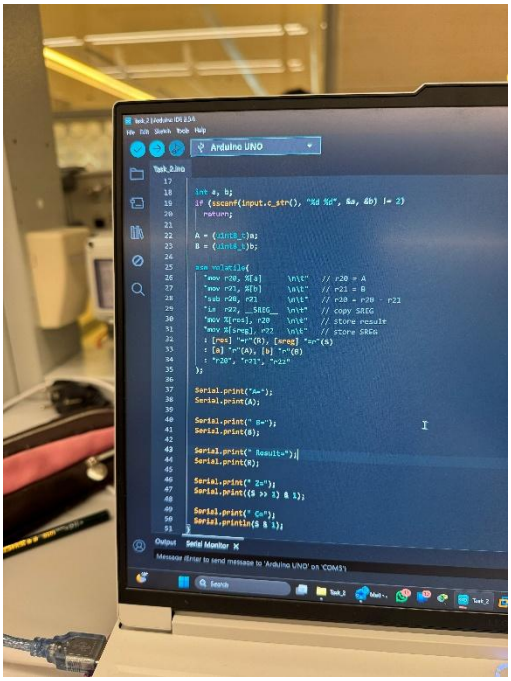


Figure 8 : Full code for the subtraction operation on the serial interface using the CPU registers only, showing the flag extraction and Serial.print() output logic

Why Carry Flag Is Behaving Like a Borrow Flag During Subtraction

The AVR microcontrollers are based on the RISC architecture, on which the unsigned subtraction is implemented as an addition of the two's complement. The "sub" instruction computes A + (NOT B/B') + 1, if in this operation does not produce the Carry Out (0) of MSB (the most significant bit), it means A<B and the CPU takes this 0 and inverts it to 1 and sets the Carry flag (Borrow).

Operation	Condition	MSB Carry Out	SREG C Flag
Add	A + B > 255	1	1
Add	A + B <= 255	0	0
Sub	A < B	0	1
Sub	A >= B	1	0

The Output Process Shown Step-by-Step

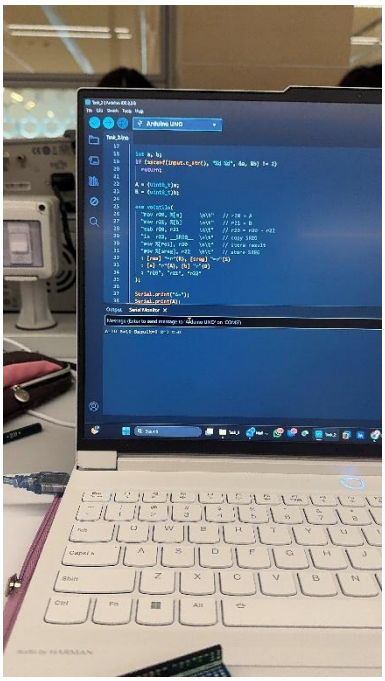


Figure 9: First test case result: A=10 B=10 Result=0 Z=1 C=0

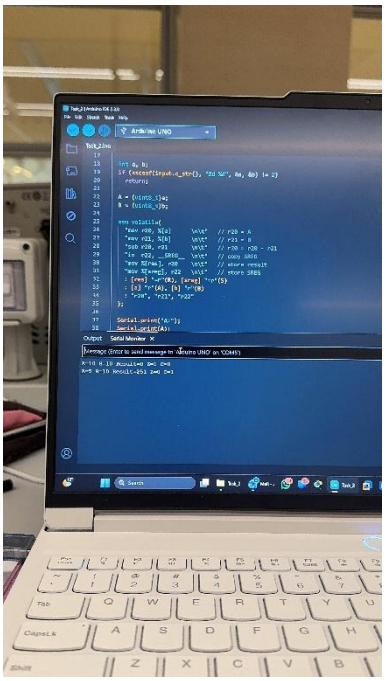


Figure 10: Two test cases results: A=10 B=10 Result=0 Z=1 C=0 | A=5 B=10 Result=251 Z=0 C=1

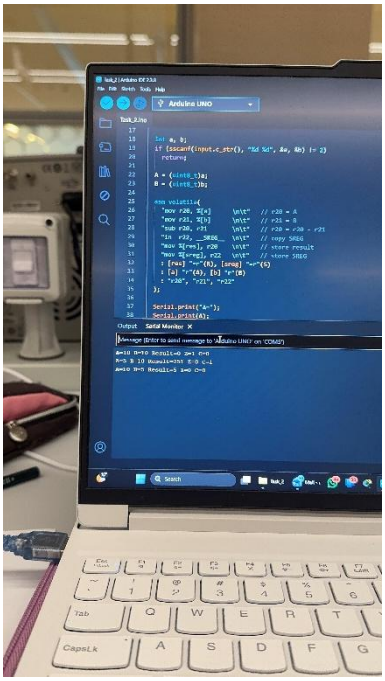


Figure 11: Three test cases results: A=10 B=10 Result=0 Z=1 C=0 | A=5 B=10 Result=251 Z=0 C=1 | A=10 B=5 Result=5 Z=0 C=0

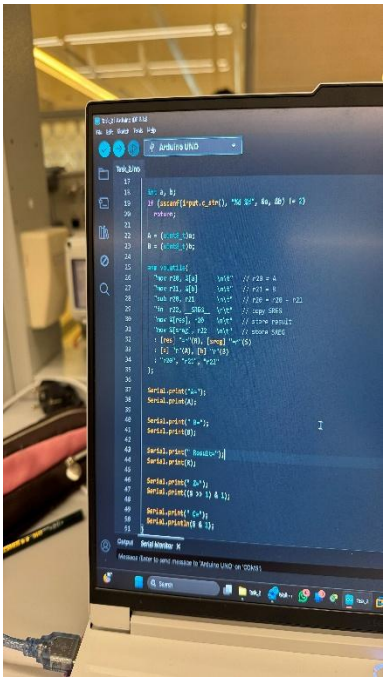


Figure 12 : Inline assembly code, the active Serial Monitor as well as the Serial.print() output logic

Task 3: *Using AVR language, implement a counter that increments a single register by 1 every 1 second. Identify the number of CPU cycles and instructions that that place within this window for your code. Implement a serial command that allows you to store or reset the current counter value in the EEPROM. Unplug Arduino. On power-up, the counter should continue from the last stored value rather than starting over (unless reset). Explain any anomalies or limitations that may occur. Indicate the address location of the register you are using to increment a value.*

Task Overview

The objective of this task is to create an 8-bit counter which increments register by 1 every one-second interval using the inline AVR assembly language. The current value of the counter is printed to the Serial Monitor, and two commands allow to save the current count value to the EEPROM address 0X01 (using command 'S') or reset it to zero (using command 'Z'). On power-up, the counter reads the EEPROM address 0x00 and continues from the saved value, which means the counter keeps the value unless specifically reset.

Register and Memory Map

Location	Address	Contains	Purpose
CPU Register r16	0x10	Current counter value (0-255)	Register, used in the inline asm instruction, is in the upper register file and executes all AVR instructions directly.
EEPROM 0x00	0x00	Reset triggered	The counter starts at 0 after 'R' command.
EEPROM 0x01	0x01	Saved counter value	Writes the current value on the 'S' command and continues the count. On power-up continues from the last saved counter value.

How the Code Logic Works

In setup(), the sketch reads EEPROM address 0x00 via the manual EEPROM driver (which polls the EEPE busy bit before every access). If the byte is 0xFF — the factory-erased default — it treats this as an uninitialized EEPROM and starts the counter at 0. Otherwise it loads the saved value into myCounter. This means the very first power-up always starts at 0, and subsequent power-ups resume from the last 'S' command. In loop(), three inline assembly instructions perform the increment directly on the CPU register r16. LDS (Load Direct from SRAM) copies

myCounter from SRAM into r16, INC adds 1 to r16 in a single cycle, and STS (Store Direct to SRAM) writes r16 back to myCounter. The C++ code then prints the value and checks for serial inputbefore calling delay(1000) to create the 1-second window.

CPU Cycle and Instruction Count within the 1-second Window

The ATmega328P runs at 16 MHz, so one clock cycle is 62.5 ns and the processor completes exactly16,000,000 cycles in one second. I found it useful to break down what actually happens inside that 1-second window because it shows just how little of it the assembly counter uses. The three assembly instructions together cost only 5 cycles -- 312.5 ns -- while delay(1000) consumes the remaining ~15.99 million.

Why counter is limited to 255

The counter uses a 'volatile uint8_t', which is an unsigned 8-bit integer capable of holding values only from 0 to 255. When the counter value is at 255, the 8-bit register can not reach 256 (in binary 100000000) as it is a 9-bit value. The 8-bit register silently wraps the counter value back to 0 (in binary 00000000). This is the standard overflow behaviour in an 8-bit counter, it does not raise any error notifications or set the Carry flag (unlike 'add' operation on the overflow). This limitation could be overcome by using either uint16_t or uint32_t, two options with 16 and 32-bit unsigned, which can increase the counter range, but will take more memory in EEPROM bytes to store.

Task 4*: Build an interrupt-driven system where pressing a button connected to D2 turns an output pin ON for exactly 10 milliseconds and then turns it OFF automatically. Configure Timer1 to generate an interrupt every 1 millisecond, and use this interrupt to control a 10 ms countdown (i.e., interrupt would trigger 10 times). Configure the external interrupt on D2 so that when the button is pressed, it immediately sets the output pin HIGH and starts the countdown. Each 1 ms timer interrupt should reduce the countdown value, and when it reaches zero, the output pin must be set LOW. The main loop must remain empty, and all behaviour must be handled using direct AVR register configuration without Arduino helper functions. You must calculate how to achieve the 1 ms timer interval, predict the 10 ms pulse width, measure the actual pulse using an oscilloscope, and compare the measured result with your calculated expectation.*

Task Overview

This task was about building an interrupt-driven system, where all behaviour was managed using directly the AVR registers and the main loop remained empty doing nothing. When a button wired to D2 (INT0-External Interrupt 0) was pressed, triggers an external interrupt that would set the onboard LED HIGH for exactly 10 seconds and at the same time set a countdown of 10 ms. Simultaneously, Timer1 runs and generates an interrupt every 1 ms, each of those 1 ms ticks would decrement the countdown and when it reaches zero the LED goes LOW automatically. The result is a 10 ms pulse with 1 ms interval without the call of delay() function.

Timer1 CTC Calculation

Timer1 is a 16-bit timer on the ATmega328P microcontroller. In CTC (Clear Timer on Compare) mode, the timer starts counting from 0 up to the tick value in OCR1A register, then resets and triggers the TIMER1_COMPA_vect, an interrupt vector. The formula for compare value is:

$$OCR1A = \frac{CPU\ Frequency}{Prescaler\ *\ Interrupt\ Frequency} - 1$$

Our Interrupt Frequency is 1 ms period, which is 1000 Hz, Prescaler value is 64 (CS00 = 1 and CS01 = 1 from *Table 14-9*) and CPU Frequency value equals 16 MHz = 16000000 Hz. We decreased the value we got by 1, because the Timer1 in CTC mode starts counting from 0, which means the match value is included in the count. By implementing the formula, we get OCR1A = (16000000 / (64 * 1000)) – 1 => 250 – 1 = 249. OCR1A = 249 provides a perfect 1 ms tick without error.

Table 14-9. Clock Select Bit Description

CS02	CS01	CS00	Description
0	0	0	No clock source (Timer/Counter stopped)
0	0	1	$clk_{I/O}$/(no prescaling)
0	1	0	$clk_{I/O}$/8 (from prescaler)
0	1	1	$clk_{I/O}$/64 (from prescaler)
1	0	0	$clk_{I/O}$/256 (from prescaler)
1	0	1	$clk_{I/O}$/1024 (from prescaler)
1	1	0	External clock source on T0 pin. Clock on falling edge.
1	1	1	External clock source on T0 pin. Clock on rising edge.

The desired output period is defined by another formula:

$$T_{OUT} = \frac{Prescaler \times TicksCount}{CLK_{Freq}}$$

T(OUT) is the time period for one cycle of the Timer1, it equals the value of prescaler (64) multiplied by TicksCount (it includes OCR1A value + 1, number of ticks per cycle). And divided by CLK(Freq), which is the CPU clock frequency (16 MHz). We get (64 * (249 + 1)) / 16000000 = 16000 / 16000000 = 0.001 => 1 ms exactly.

Register Configuration

For this task we need to configure different types of registers for the input pin, output pin, external interrupt, timer and pull-up register directly writing to the register itself.

Register	Bits set	Code format	Process
DDRD	DDD2 set 0 (cleared)	DDRD &= ~(1 << DDD2);	Set pin D2 (INT0) as input
PORTD	PORTD2 set 1	PORTD |= (1 << PORTD2);	Enable internal pull-up resistor on pin D2, so it will go HIGH when the button is not pressed
DDRB	DDB5 set 1	DDRB |= (1 << DDB5);	Sets PB5 (pin D13, the built-in LED) as an output
EICRA	ISC01 set 1 ISC00 set 0 (cleared)	EICRA |= (1 << ISC01);	Configure INT0 to trigger on the falling edge (read LOW when the button is pressed, meaning connected to GND)
EIMSK	INT0 set 1	EIMSK |= (1 << INT0);	Enable the external interrupt INT0 (pin D2)
TCCR1B	WGM12 set 1 CS11 + CS10 set 1	TCCR1B |= (1 << WGM12); TCCR1B |= (1 << CS11) | (1 << CS10); // prescaler	CTC mode and prescaler bit selection
OCR1A	All 16 bits set	249	This is the compare value, when reaching it the timer will reset and trigger an interrupt
TIMSK1	OCIE1A set 1	TIMSK1 |= (1 << OCIE1A);	Enable Timer/Counter1 Interrupt Mask

Oscilloscope Evidence

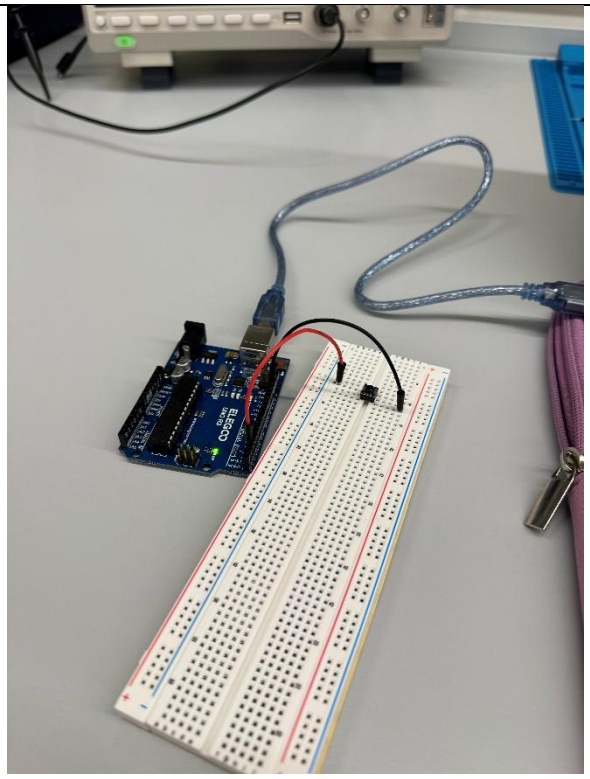


Figure 13: Arduino UNO R3 connected to the breadboard with a button wired to pin D2 for interrupt-driven pulse lab task

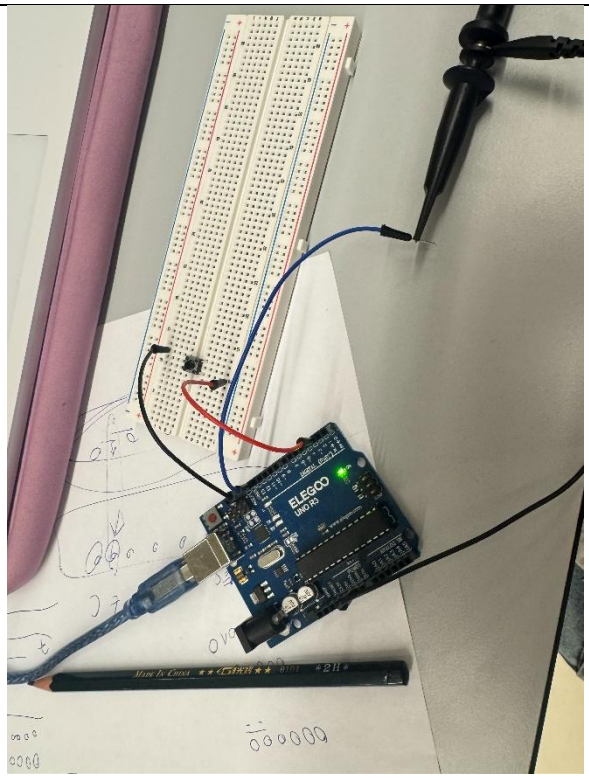


Figure 14: The oscilloscope probe connected to the output pin (Onboard LED pin D13) with the Arduino UNO R3 and breadboard layout

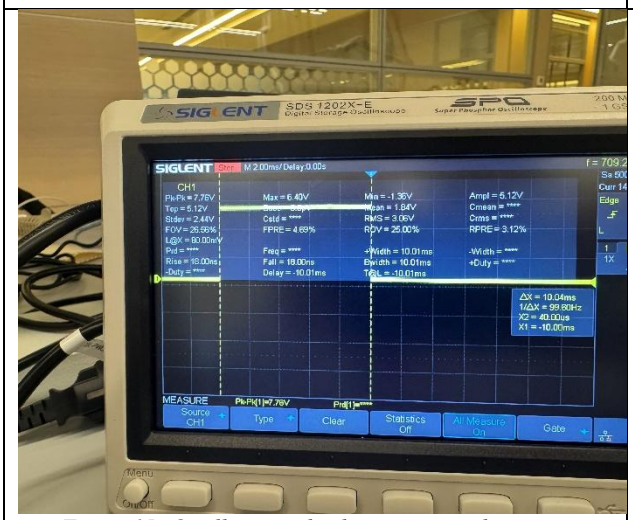


Figure 15: Oscilloscope display capturing the output waveform on pin D13

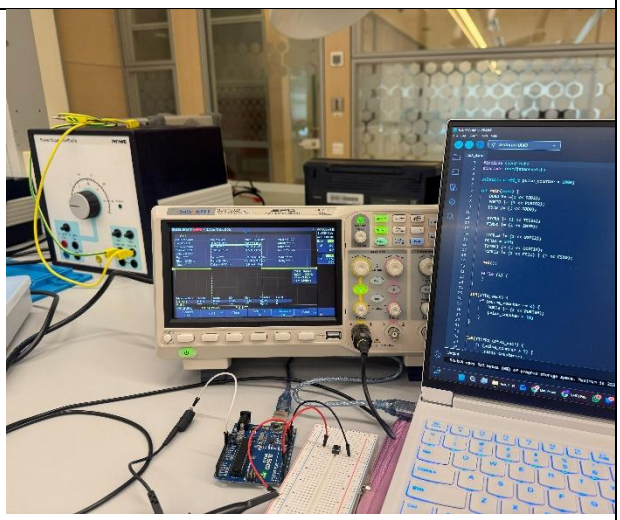


Figure 16: Full hardware and software setup for the lab 4

Task 5: *AVR CONTROL FLOW LAB: RJMP VS JMP VS IJMP ON ARDUINO UNO (ATMEGA328P)*

Hardware: Arduino Uno (ATmega328P) | LED: Built-in LED on D13 (PB5) | Button: D8 (PB0) to GND (use internal pull-up)

Objective: Build one firmware that behaves like a simple "one-button LED gadget" and demonstrates three different jump styles:

• RJMP: tight local polling loop at boot

• JMP: hard jump into one of three modes (Mode A/B/C)

• IJMP: runtime dispatch to one of several "actions" inside the selected mode

Final behaviour: On power-up, the device waits for the first button press, then confirms with a short LED flash. During the next 2 seconds, the number of presses selects a mode:

• 1 press = Mode A (slow blink)

• 2 presses = Mode B (double blink)

• 3 presses = Mode C (fast strobe)

While running in a mode, each short press cycles an "action" (0..3). The action is executed using a jump table and IJMP.

Phase requirements:

• Phase 1 (RJMP): Implement the boot wait loop in assembly using a label, a bit-test (SBIC or SBIS), and rjmp back to the label while the button is not pressed.

• Phase 2 (JMP): After counting presses, enter the chosen mode by using an explicit jmp modeX_entry. The mode entry functions run forever and do not return.

• Phase 3 (IJMP): Inside the mode loop, implement action dispatch by loading a handler address into Z (r31:r30) and executing ijmp. Each handler must jump back to the mode loop (since IJMP does not push a return address).

Task Overview

In this task we implement one firmware which demonstrates all three different jump styles (RJMP, JMP, IJMP). On power-up the device waits for a first button press, which uses RJMP loop. After a confirmation flash, theres a 2-second window where the firmware counts the number of button presses for three LED modes (A, B or C). Then using JMP it enters the according mode permanently. Inside of each mode, short button presses trigger an action using IJMP.

Phase Comparison of Jump Instructions

Jump Type	Encoding	Range	Cycles	Description	Use for this Task
RJMP	1 word / 16 bits	-2048 - +2047 words from PC	2 (shorter fetch)	Relative Jump: PC ← PC + k + 1 (offset small)	Boot wait loop and return from action() to mode loops.
JMP	2 words / 32 bits	0 to 4 M words (entire flash memory)	3 (longer fetch)	Absolute Jump: PC = k (Full program memory address)	Mainly used for the selection of the modes A, B or C. Also, used in action().
IJMP	1 word / 16 bits	0 to 64K words (Z register)	2	Indirect Jump: PC = Z (loads from register Z instead of an immediate address)	Used for the dispatch of action()

Phase 1 – RJMP Boot Wait Loop

For the initial boot wait RJMP is used as it is the most efficient choice for a tight loop (instead of JMP with a longer fetch). The SBIC (Skip if Bit in I/O register is Cleared) instruction tests one bit in I/O register, if it is set, meaning the pin is HIGH while the button is not pressed, SBIC does not skip the RJMP and executes the branch instruction and continues. However, when the button is pressed and the pin goes LOW, the bit is cleared (0) and SBIC skips the RJMP continuing with the next instruction.

Phase 2 – JMP Mode Dispatch

The firmware uses the JMP instruction to enter the modes after 2 second wait, because JMP can encode a full 22-bit absolute address, this guarantees that it will reach any address in the ATmega328P's 32 KB of flash memory regardless of where the mode functions are placed. This is not true for RJMP, since it will stop with an error if the instruction attempts to target a destination more than 2047 words away from the current word being executed. The mode functions themselves are declared noreturn (meaning they never return to the main program) therefore, the use of the JMP instruction to enter modes is more suitable since it represents a one-way transfer of control rather than a call to a function.

Phase 3 – IJMP Action Dispatch

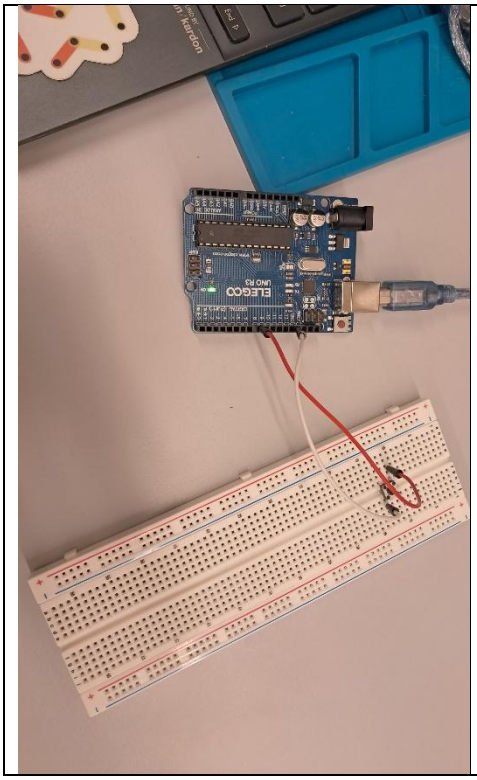
Within each of these modes, the action handlers are dispatched to one of four possible action handlers based on the value of their action_index. The jump instruction for jumping to these

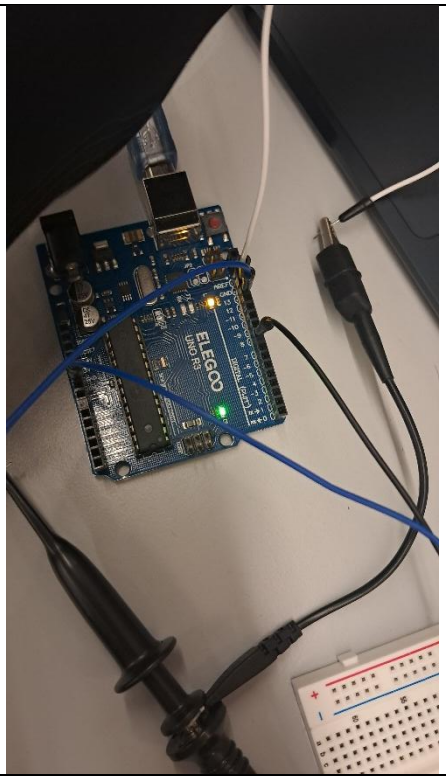
action handlers is IJMP, while running the function pointer corresponding to the current action is loaded into the Z register (r31:r30) via the ‘movw’ (Move Word) instruction and then control is transferred to that address with the IJMP instruction. In contrast with ‘icall’ (Indirect Call), IJMP does not push a return address onto the stack, so each action handler has to explicitly jump back to the mode_loop label using either RJMP (Relative Jump) or JMP (Jump).

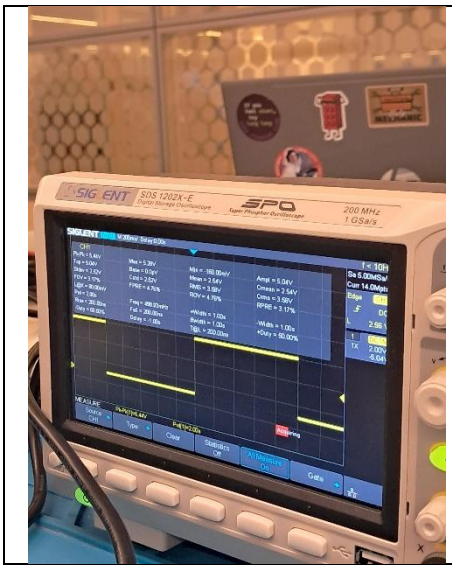
Mode and Action Summary

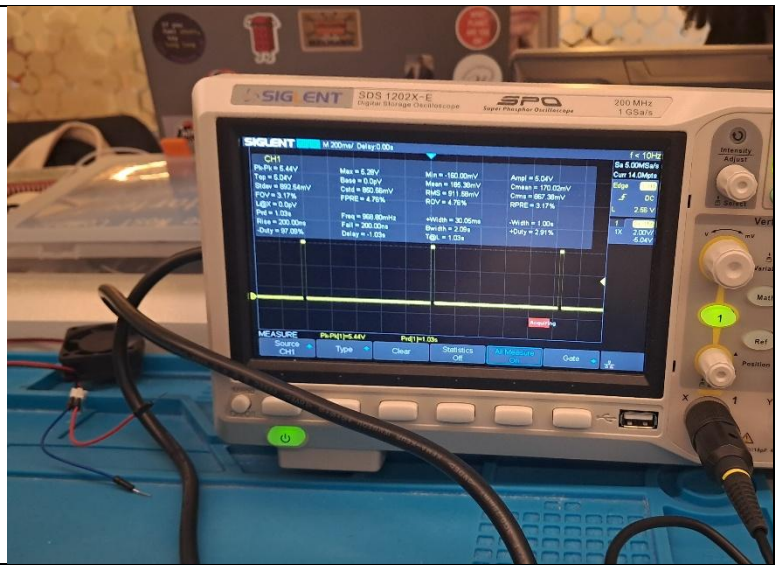
Mode	Entry Jump	Presses	g_on_time	g_off_time	Behaviour
A – Slow Blink	jmp modeA_entry	1 (default)	1000 ms	1000 ms	LED on for 1 s
B – Double Blink	jmp modeB_entry	2	120 ms	120 ms	Two quick 120 ms flashes
C – Fast strobe	jmp modeC_entry	3+	50 ms	50 ms	Fast 50 ms on/off strobe

Oscilloscope Evidence






SIG ENT
SDS 1202X-E
SPO
200 MHz
1 GSa/s


SIG ENT
SDS 1202X-E
SPO
200 MHz
1 GSa/s

Reference Page

Arduino Due digitalWrite vs. direct port manipulation speed - Mega / Due - Arduino Forum

digitalWrite() | Arduino Documentation

Secrets of Arduino PWM | Arduino Documentation

UNO R3 | Arduino Documentation

Oscilloscope Basics | Reading & Operating Tutorial | Tektronix

AVR Tutorials - The Status Register

Arduino Timers [Ultimate Guide]